

Java Game Development Tutorial Part 3

JApplets and Swing

While we have been using the AWT (Abstract Windowing Toolkit) and it has been part of Java from the beginning, it is not powerful or flexible enough for writing complex, sophisticated applications so I want to introduce you to Swing.

The Swing graphical user interface library was created to address the problems with the AWT. The classes that make up the Swing library can be found in the package `javax.swing`. Swing includes the class `javax.swing.JApplet` to be used as a basis for writing applets.

JApplet is actually a subclass of Applet, so JApplets are in fact Applets in the usual sense. However, JApplets have a lot of extra structure that plain Applets don't have. Because of this structure, the painting of a JApplet is a more complex affair and is handled by the system. So, when you make a subclass of JApplet you should **not** write a `paint()` method for it. As we will see, if you want to draw on a JApplet, you should add a component to the applet to be used for that purpose. On the other hand, you **can** and generally should write an `init()` method for a subclass of JApplet.

Let's take a look at a simple JApplet that uses Swing. This applet demonstrates some of the basic ideas of GUI programming. Although you won't understand everything in it at this time, it will give you a preliminary idea of how things work.

GUI programs use "components" such as buttons to allow interaction with the user. Our sample applet contains a button. In fact, the button is the only thing in the applet, and it fills the entire rather small applet. Here's our sample JApplet, which is named Hello:

Here is the source code copy it and give it a try!

```
import javax.swing.*; // Swing GUI classes are defined here.
import java.awt.event.*; // Event handling class are defined here.
```

```
public class Hello extends JApplet {

    public void init() {

        JButton btn = new JButton("Click Me!");

        btn.addActionListener( new ActionListener() {
            // The "action listener" for the button is defined
            // by this nested anonymous class.

            public void actionPerformed(ActionEvent evt) {

                // This method is called to respond when the user
                // presses the button. It displays a message in
                // a dialog box, along with an "OK" button which
```

```

        // the user can click to dismiss the dialog box.

        String title = "Greetings"; // Shown in box's title bar.
        String message = "Another hello from Swing.";
        JOptionPane.showMessageDialog(null, message, title,
                                    JOptionPane.INFORMATION_MESSAGE);
    }
});

getContentPane().add(btn);

}

}

```

Notice that I use some of the same ways to inheritance structure as before. This time I use `public class Hello` extends **JApplet** instead of `public class Something` extends **Applet**

Some other new features:

The button:

The button this applet is an object that belongs to the class `JButton` (more properly, `javax.swing.JButton`). When the applet is created, the button must be created and added to the applet. This is part of the process of initializing the applet and is done in the applet's `init()` method. In this method, the button is created with the statement:

```
JButton btn = new JButton("Click Me!");
```

The parameter to the constructor specifies the text that is displayed on the button. The button does not automatically appear on the screen. It has to be added to the applet's "content pane." This is done with the statement:

```
getContentPane().add(btn);
```

Once it has been added to the applet, a `JButton` object mostly takes care of itself. In particular, it draws itself, so you don't have to worry about drawing it. When the user clicks the button, it generates an event. The applet (or, in fact, any object) can be programmed to respond to this event. Event-handling is the major topic in GUI programming, and I will cover it in detail later. But in outline, it works like this: The type of event generated by a button is called an `ActionEvent`. For the applet to respond to an event of this type, it must define a method

```
public void actionPerformed(ActionEvent evt) { . . . }
```

Furthermore, the button must be told that the applet will be "listening" for action events from the button. This is done by calling one of the button object's instance methods, `addActionListener()`, in the applet's `init()` method.

What should the applet do in its `actionPerformed()` method? When the user clicks the button, we want a message window to appear on the screen. Fortunately, Swing makes this easy. The class `swing.javax.JOptionPane` has a static method named `showMessageDialog()` that can be used for this purpose, so all we have to do in `actionPerformed()` is call that method.

Now that we have looked at how to implement JApplet let's get some more experience with the paintComponent

Let's use some of the material covered in the first tutorial that looked at some graphics to write a JApplet that paints multiple words.

The applet will draw multiple copies of a message on a black background. Each copy of the message is in a random color. Five different fonts are used, with different sizes and styles. The displayed message is the string "Java!", but a different message can be specified in an applet param.

The source for the applet is shown below. I use an instance variable called `message` to hold the message that the applet will display. There are five instance variables of type `Font` that represent different sizes and styles of text. These variables are initialized in the applet's `init()` method and are used in the drawing surface's `paintComponent()` method. I also use the `init()` method to create the drawing surface, add it to the applet, and set its background color to black.

The `paintComponent()` method for the drawing surface simply draws 25 copies of the message. For each copy, it chooses one of the five fonts at random, and it calls `g.setFont()` to select that font for drawing the text. It creates a random HSB color and uses `g.setColor()` to select that color for drawing. It then chooses random (x,y) coordinates for the location of the message. The x coordinate gives the horizontal position of the left end of the string. The formula used for the x coordinate, `"-50 + (int)(Math.random()*(width+40))"` gives a random integer in the range from -50 to width-10. This makes it possible for the string to extend beyond the left edge or the right edge of the applet. Similarly, the formula for y allows the string to extend beyond the top and bottom of the applet.

The drawing surface class, which is named `Display`, defines the `paintComponent()` method that draws all the strings that appear in the applet. The drawing surface is created in the applet's `init()` method as an object of type `Display`.

Here is the complete source code for the `RandomStrings` applet:

```
import java.awt.*;
```

```

import javax.swing.*;

public class Text extends JApplet {

    String message; // The message to be displayed. This can be set in

        // an applet param with name "message". If no

        // value is provided in the applet tag, then

        // the string "Java!" is used as the default.


    Font font1, font2, font3, font4, font5; // The five fonts.


    Display drawingSurface; // This is the component on which the

        // drawing will actually be done. It

        // is defined by a nested class that

        // can be found below.


    public void init() {

        // Called by the system to initialize the applet.

        message = getParameter("message");

        if (message == null)

            message = "Java Rocks!";


        font1 = new Font("Serif", Font.BOLD, 14);

        font2 = new Font("SansSerif", Font.BOLD + Font.ITALIC, 24);

        font3 = new Font("Monospaced", Font.PLAIN, 20);

        font4 = new Font("Dialog", Font.PLAIN, 30);

```

```
font5 = new Font("Serif", Font.ITALIC, 36);
```

```
drawingSurface = new Display(); // Create the drawing surface.
```

```
drawingSurface.setBackground(Color.black);
```

```
setContentPane(drawingSurface); // Since drawingSurface will fill
```

```
    // the entire applet, we simply
```

```
    // replace the applet's content
```

```
    // pane with drawingSurface.
```

```
} // end init()
```

```
class Display extends JPanel {
```

```
    // This nested class defines a JPanel that is used
```

```
    // for displaying the content of the applet. An
```

```
    // object of this class is used as the content pane
```

```
    // of the applet. Note that since this is a nested
```

```
    // non-static class, it has access to the instance
```

```
    // variables of the main class such as message and font1.
```

```
public void paintComponent(Graphics g) {
```

```
    super.paintComponent(g); // Call the paintComponent method from
```

```
    // the superclass, JPanel. This simply
```

```
    // fills the entire component with the
```

```
// component's background color.
```

```
int width = getSize().width; // Get this component's width.
```

```
int height = getSize().height; // Get this component's height.
```

```
for (int i = 0; i < 25; i++) {
```

```
    // Draw one string. First, set the font to be one of the five
```

```
    // available fonts, at random.
```

```
    int fontNum = (int)(5*Math.random()) + 1;
```

```
    switch (fontNum) {
```

```
        case 1:
```

```
            g.setFont(font1);
```

```
            break;
```

```
        case 2:
```

```
            g.setFont(font2);
```

```
            break;
```

```
        case 3:
```

```
            g.setFont(font3);
```

```
            break;
```

```
        case 4:
```

```
            g.setFont(font4);
```

```
            break;
```

```
        case 5:
```

```
            g.setFont(font5);
```

```
        break;

    } // end switch

    // Set the color to a bright, saturated color, with random hue.

    float hue = (float)Math.random();

    g.setColor( Color.getHSBColor(hue, 1.0F, 1.0F) );

    // Select the position of the string, at random.

    int x,y;

    x = -50 + (int)(Math.random()*(width+40));

    y = (int)(Math.random()*(height+20));

    g.drawString(message,x,y);

} // end for

}

}
```